

۱- ساختار PCB و همچنین وضعیتهای تعریف شده برای هر پردازش را در xv6 پیدا کرده و گزارش کنید. آیا شباهتی میان داده‌های موجود در این ساختار و ساختار به تصویر کشیده شده در شکل 3.3 منبع درس وجود دارد؟

در xv6، ساختار داده‌ای که اطلاعات هر پردازش را نگه می‌دارد، struct proc است که در فایل proc.h تعریف شده است. این ساختار معادل PCB (Process Control Block) در سیستم عامل است و شامل فیلدهایی مانند pid شناسه پردازش، state وضعیت فعلی پردازش، parent پردازش والد، kstack آدرس پشته کرنل، context ذخیره زمینه برای تعویض زمینه، chan علت خواب پردازش و cwd دایرکتوری جاری می‌شود. وضعیت‌های تعریف شده برای پردازش‌ها در xv6 عبارتند از:

- **UNUSED:** پردازش استفاده نشده.
- **EMBRYO:** پردازش در حال ایجاد.
- **SLEEPING:** پردازش در خواب (منتظر رویداد).
- **RUNNABLE:** پردازش آماده اجرا.
- **RUNNING:** پردازش در حال اجرا.
- **ZOMBIE:** پردازش خاتمه یافته اما هنوز PCB آن آزاد نشده است.

این ساختار و وضعیت‌ها شباهت قابل توجهی به ساختار ارائه شده در شکل 3.3 منبع درس دارند. در آن شکل نیز فیلدهایی مانند وضعیت پردازش، شماره برنامه، رجیسترها، اطلاعات مدیریت حافظه و ... وجود دارد که در ساختار proc نیز مشاهده می‌شود. با این حال، xv6 نسخه ساده‌تری از PCB را پیاده‌سازی کرده و برخی جزئیات پیشرفته‌تر (مانند مدیریت اولویت یا زمان‌بندی پیچیده) در آن وجود ندارد.

2- برای هر وضعیت پردازش در xv6 معادل وضعیت مربوطه در شکل ا را مشخص کنید.

در شکل ۱ معمولاً یک دیاگرام حالت‌های پردازش نمایش داده می‌شود که شامل حالت‌هایی مانند جدید (New)، آماده (Ready)، در حال اجرا (Running)، منتظر (Waiting) و خاتمه (Terminated) است. معادل این حالت‌ها در xv6 به شرح زیر است:

- جدید EMBRYO ↔ (New) پردازش در حال ایجاد است.
- آماده RUNNABLE ↔ (Ready) پردازش آماده اجرا است اما در حال اجرا نیست.
- در حال اجرا RUNNING ↔ (Running) پردازش در حال اجرا روی CPU است.
- منتظر SLEEPING ↔ (Waiting) پردازش منتظر رویداد یا منبع است.
- خاتمه ZOMBIE ↔ (Terminated) پردازش پایان یافته اما PCB آن هنوز موجود است.

وضعیت UNUSED در xv6 معادل حالت اولیه (عدم وجود پردازش) در نظر گرفته می‌شود و در دیاگرام کلاسیک حالت‌ها به طور صریح نمایش داده نمی‌شود. این تطابق نشان می‌دهد که xv6 از یک مدل ساده اما کامل برای مدیریت حالت‌های پردازش پیروی می‌کند.

3- فرآیند کامل ایجاد یک پردازش جدید در xv6 و نحوه گذار آن از حالت new به حالت ready را شرح دهید. با توجه به توضیحات چرخه وضعیت پردازش‌ها و ساختار xv6، مشخص کنید کدام توابع موجود در proc.c باعث گذار یک

پردازه از حالت **new** به حالت **ready** در شکل ۱ می‌شوند. در **xv6**، این گذار از چه حالت/حالت‌هایی به چه حالت/حالت‌هایی تغییر می‌کند؟

در **xv6**، ایجاد یک پردازه جدید عمدتاً از طریق تابع **fork()** یا در مرحله اولیه بوت از طریق **userinit()** انجام می‌شود. هسته این فرآیند در تابع **allocproc()** در فایل **proc.c** قرار دارد که مسئول اختصاص و مقداردهی اولیه ساختار **proc (PCB)** است. این تابع ابتدا وضعیت پردازه را به **EMBRYO** معادل **new** تنظیم می‌کند، سپس پشته کرنل، زمینه (**context**) و دیگر فیلدهای ضروری را آماده می‌سازد. پس از تکمیل مقداردهی، پردازه با تنظیم وضعیت به **RUNNABLE** معادل **ready** به صف پردازه‌های آماده اجرا اضافه می‌شود.

توابع کلیدی درگیر در این فرآیند:

- **allocproc()** ایجاد ساختار پردازه و مقداردهی اولیه آن وضعیت اولیه **EMBRYO**.
- **fork()** کپی کردن پردازه والد به فرزند و در نهایت تغییر وضعیت فرزند به **RUNNABLE**.
- **userinit()** ایجاد اولین پردازه کاربر (**init**) در راه‌اندازی سیستم.
- **scheduler()** انتخاب پردازه‌های **RUNNABLE** و تغییر وضعیت آن‌ها به **RUNNING** هنگام تخصیص CPU.

گذار حالت‌ها در **proc.c**:

در **xv6**، گذار از وضعیت **EMBRYO** (جدید) به **RUNNABLE** (آماده) عمدتاً در پایان توابع **fork()** و **userinit()** اتفاق می‌افتد. به طور مشخص، پس از تکمیل موفقیت‌آمیز مراحل ایجاد پردازه، دستور **p->state = RUNNABLE** اجرا می‌شود. این تغییر حالت، پردازه را به صف اجرا منتقل می‌کند و پس از آن توسط زمان‌بند (**scheduler**) می‌تواند انتخاب شده و به حالت **RUNNING** برود. در مقابل، اگر خطایی در فرآیند ایجاد رخ دهد، وضعیت پردازه به **UNUSED** بازمی‌گردد و منابع آن آزاد می‌شود. این مکانیسم ساده اما مؤثر، گذار کلاسیک از "جدید" به "آماده" را در چرخه حیات پردازه پیاده‌سازی می‌کند.

4-سقف تعداد پردازه‌های ممکن در xv6 چه عددی است؟ در صورتی که یک پردازه تعداد زیادی پردازه فرزند ایجاد کند و از این سقف عبور کند، کرنل چه واکنشی نشان داده و برنامه سطح کاربر چه بازخوردی دریافت می‌کند؟

بله، در **xv6** یک محدودیت ثابت برای حداکثر تعداد پردازه‌های هم‌زمان وجود دارد. این مقدار در فایل **param.h** با نام **NPROC** تعریف شده است که معمولاً برابر ۶۴ پردازه است. این عدد شامل تمام پردازه‌های در حالت‌های مختلف (**UNUSED**, **EMBRYO**, **SLEEPING**, **RUNNABLE**, **RUNNING**, **ZOMBIE**) می‌شود.

ب) واکنش کرنل هنگام عبور از سقف:

هنگامی که یک پردازه سعی کند با استفاده از فراخوانی سیستمی **fork()** پردازه فرزند جدیدی ایجاد کند، کرنل ابتدا در تابع **allocproc()** در فایل **proc.c** به دنبال یک **slot** خالی با وضعیت (**UNUSED**) در جدول پردازه‌ها می‌گردد. اگر همه **slot**ها پر باشند (یعنی تعداد پردازه‌های فعال به **NPROC** رسیده باشد)، **allocproc()** نمی‌تواند پردازه جدیدی اختصاص دهد و مقدار **NULL** برمی‌گرداند. در ادامه، فراخوانی سیستمی **fork()** با شکست مواجه شده و خطا باز می‌گرداند.

ج) بازخورد به برنامه سطح کاربر:

برنامه کاربردی که **fork()** را فراخوانی کرده، مقدار بازگشتی **-1** را دریافت می‌کند و **errno** روی مقدار **EAGAIN** یا معادل آن در **xv6** معمولاً به صورت مستقیم **-1** بدون تنظیم **errno** خاصی باز می‌گردد تنظیم می‌شود. بدین ترتیب برنامه

می‌تواند با بررسی مقدار بازگشتی `fork()`، تشخیص دهد که ایجاد پردازنده فرزند با شکست مواجه شده و اقدامات مناسب (مانند انتظار برای خالی شدن منابع یا خروج کنترل‌شده) را انجام دهد.

د) محل بررسی این محدودیت در `xv6`:

این بررسی عمدتاً در تابع `allocproc()` انجام می‌شود. این تابع در جدول پردازنده‌ها (`proc[]`) به دنبال اولین پردازنده با وضعیت `UNUSED` می‌گردد. اگر هیچ `slot` خالی پیدا نکند، عملیات ناموفق خواهد بود.

5- چرا نیاز است در ابتدای هر حلقه تابع `scheduler` جدول پردازنده‌ها قفل شود؟ آیا در سیستم‌هایی که تنها یک پردازنده (یک هسته پردازشی) دارند هم نیاز است این کار صورت بگیرد؟

در `xv6`، تابع `scheduler` که در هر هسته اجرا می‌شود، مسئول انتخاب و اجرای پردازنده‌های `RUNNABLE` است. قفل کردن جدول پردازنده‌ها (`ptable.lock`) در ابتدای حلقه اصلی این تابع به دلایل زیر ضروری است:

1. حفاظت از ساختارهای داده اشتراکی:
جدول پردازنده‌ها (`ptable.proc`) یک ساختار داده اشتراکی بین تمام هسته‌ها است. قفل از دسترسی همزمان چند هسته به این جدول و تغییر رقابتی (`race condition`) وضعیت پردازنده‌ها مانند تغییر حالت از `RUNNABLE` به `RUNNING` جلوگیری می‌کند.
2. انسجام در پیمایش و انتخاب پردازنده:
بدون قفل، ممکن است یک هسته در حال پیمایش جدول برای انتخاب پردازنده باشد و هسته دیگر همزمان پردازنده‌ای را حذف یا وضعیت آن را تغییر دهد که منجر به رفتار غیرقابل پیش‌بینی می‌شود.
3. هماهنگی با وقفه‌ها:
حتی در سیستم‌های تک‌هسته‌ای، وقفه‌ها مانند وقفه تایمر یا دستگاه‌های I/O می‌توانند در هر نقطه‌ای از کد کرنل، از جمله در وسط تابع `scheduler`، رخ دهند. اگر کنترل به دست روال وقفه‌افزار (`interrupt handler`) بیفتد، ممکن است وضعیت پردازنده‌ها تغییر کند (مثلاً پردازنده‌ای از حالت `SLEEPING` به `RUNNABLE` برود). قفل `ptable.lock` در زمان وقفه نیز حفظ می‌شود تا از ناسازگاری داده‌ها جلوگیری شود.

پس حتی در سیستم تک‌پردازنده نیز این قفل‌گذاری لازم است زیرا وقفه‌ها می‌توانند به طور ناهمگام (`asynchronously`) ساختار پردازنده‌ها را تغییر دهند و بدون مکانیزم قفل، ممکن است `scheduler` داده‌های ناسازگاری را مشاهده کند.

6- با فرض اینکه `xv6` در حالت چند پردازنده‌ای (چند هسته‌ای) در حال اجراست، اگر یک پردازنده آماده به اجرا شود، و صف پردازنده‌ها در پردازنده دیگری در حال طی شدن باشد (`proc:335`) در مکانیزم زمان‌بندی `xv6` نسبت به موقعیت پردازنده در صف، در چه `iteration` ای امکان `schedule` پیدا می‌کند؟

در `xv6` چند پردازنده‌ای، هر هسته به طور مستقل تابع `scheduler` را اجرا می‌کند و پردازنده‌های `RUNNABLE` را از جدول پردازنده‌ها انتخاب می‌کند. `xv6` از صف‌های جداگانه برای هر هسته استفاده نمی‌کند، بلکه یک جدول مرکزی دارد. اگر پردازنده A در لحظه‌ای `RUNNABLE` شود، و همزمان هسته B در حال پیمایش جدول پردازنده‌ها (مثلاً در خط ۳۳۵ فایل `proc.c` که مربوط به حلقه `for` در `scheduler` است) باشد، دو حالت پیش می‌آید:

1. اگر پردازنده A قبل از اینکه هسته B به موقعیت آن در جدول برسد، `RUNNABLE` شده باشد:
در این حالت، چون هسته B هنوز به آن نقطه از جدول نرسیده، پردازنده A را در همان تکرار (`iteration`)

جاری از حلقه پیمایش مشاهده کرده و ممکن است آن را برای اجرا انتخاب کند (در صورتی که شرایط دیگر مانند قفل‌ها اجازه دهد).

2. اگر پردازش A پس از عبور هسته B از موقعیت آن در جدول، RUNNABLE شده باشد: در این حالت، پردازش A در تکرار جاری دیده نخواهد شد، زیرا هسته B قبلاً از آن بخش از جدول عبور کرده است. بنابراین، پردازش A باید تا تکرار بعدی از حلقه scheduler منتظر بماند تا دوباره پیمایش از ابتدای جدول شروع شود.

نتیجه‌گیری:

احتمال schedule شدن پردازش جدید بستگی به زمان دقیق RUNNABLE شدن آن نسبت به موقعیت پیمایش هسته دیگر دارد. اگر قبل از رسیدن هسته به آن موقعیت در جدول باشد، در همان iteration؛ در غیر این صورت در iteration بعدی فرصت اجرا پیدا می‌کند. این رفتار به دلیل الگوی پیمایش ساده (scan) جدول و عدم وجود صف‌های جداگانه برای هر هسته است.

7- رجیسترهای موجود در ساختار context در xv6 را نام ببرید.

ساختار context در فایل proc.h تعریف شده و برای ذخیره‌سازی وضعیت رجیسترهای پردازش در هنگام تعویض زمینه context switch استفاده می‌شود. رجیسترهای ذخیره‌شده در این ساختار عبارتند از:

- **eip: Instruction Pointer** : برنامه‌شمار
- **esp: Stack Pointer** : اشاره‌گر پشته
- **ebx: رجیستر پایه**
- **ecx: رجیستر شمارنده**
- **edx: رجیستر داده**
- **esi: رجیستر منبع**
- **edi: رجیستر مقصد**
- **ebp: Base Pointer** : اشاره‌گر پایه پشته

در معماری xv6، این رجیسترها به ترتیب در حین تعویض زمینه ذخیره و بازیابی می‌شوند تا بتوان اجرای پردازش را بعداً از همان نقطه ادامه داد.

8- همانطور که می‌دانید یکی از مهمترین رجیسترها **Program Counter** است که نشان می‌دهد روند اجرای برنامه تا کجا پیش رفته است. با ذخیره‌سازی این رجیستر می‌توان محل ادامه برنامه را بازیابی کرد. این رجیستر در ساختار context چه نام دارد؟ شیوه ذخیره و بازنشانی این رجیستر را توضیح دهید.

در ساختار context، رجیستر Program Counter با نام **eip (Extended Instruction Pointer)** ذخیره می‌شود.

شیوه ذخیره و بازنشانی: eip

1. ذخیره سازی:

در هنگام تعویض زمینه، تابع `switch` در فایل `switch.S` نوشته شده به زبان اسمبلی فراخوانی می شود. این تابع رجیستر `eip` جاری را به همراه دیگر رجیسترها روی پشته ذخیره می کند، سپس آدرس ساختار `context` پردازنده قدیم را به روز کرده و `eip` ذخیره شده در `context` پردازنده جدید را بارگذاری می کند.

2. بازنشانی:

وقتی پردازنده جدید می خواهد اجرا شود، `switch` مقدار `eip` ذخیره شده در `context` آن پردازنده را از حافظه خوانده و در رجیستر `eip` پردازنده قرار می دهد. این کار باعث می شود اجرای دستورات از آدرس بعدی که قبلاً در `eip` ذخیره شده بود، ادامه یابد.

9- همانطور که در قسمت قبل مشاهده کردید، ابتدای تابع `scheduler` وقفه به کمک تابع `sti` فعال می شود. با توجه به توضیحات این قسمت، اگر وقفه ها فعال نمی شد چه مشکل به وجود می آمد؟ آیا وقفه ای در این حالت قابلیت اجرا دارد؟ توضیح دهید.

در `xv6`، تابع `scheduler` در ابتدای حلقه اصلی خود با دستور `sti` وقفه ها را فعال می کند. اگر وقفه ها فعال نشوند، مشکلات زیر رخ می دهد:

1. از دست رفتن قابلیت پیش گیری: (Preemption)

وقفه تایمر (`timer interrupt`) که باعث فراخوانی زمان بند و امکان تعویض پردازنده می شود، نمی تواند اجرا شود. در نتیجه، اگر پردازنده ای در حالت `RUNNING` وارد حلقه بی پایان شود یا برای مدت طولانی `CPU` را اشغال کند، سیستم عملاً قفل می شود و هیچ پردازنده دیگری نمی تواند اجرا شود.

2. توقف پاسخگویی به رویدادهای خارجی:

وقفه های دستگاه های ورودی/خروجی (مانند صفحه کلید، دیسک) نیز غیر فعال می مانند. بنابراین سیستم نمی تواند به رویدادهای خارجی پاسخ دهد و پردازنده های در انتظار `I/O` ممکن است برای همیشه مسدود بمانند.

3. توقف ساعت سیستم:

متغیر `ticks` نشان دهنده زمان سیستم است، تنها توسط وقفه تایمر به روز می شود. غیر فعال بودن وقفه باعث توقف پیشروی زمان سیستم می شود.

آیا وقفه ای در این حالت قابلیت اجرا دارد؟

خیر. تا زمانی که وقفه ها غیر فعال باشند، حتی اگر وقفه ای از سخت افزار دریافت شود، پردازنده آن را نادیده می گیرد و اجرای دستورالعمل فعلی را ادامه می دهد. وقفه ها در صف می مانند تا وقتی که دوباره با دستور `sti` فعال شوند. این حالت می تواند منجر به از دست رفتن وقفه ها یا تاخیرهای غیر قابل قبول در پاسخگویی شود.

10- وقفه تایمر هر چه مدت یک بار صادر می شود؟ (راهنمایی: می توانید با اضافه کردن یک `cprintf` از `ticks++` یا با مراجعه به تابع `lapicinit` واقع در `lapic.c` این موضوع را مشاهده کنید.)

وقفه تایمر در `xv6` هر ۱۰ میلی ثانیه یکبار (معادل فرکانس ۱۰۰ هرتز) صادر می شود. این تنظیم در تابع `lapicinit` (فایل `lapic.c`) انجام می شود. در این تابع، تایمر محلی (`APIC (LAPIC)` به گونه ای پیکربندی می شود

که با فرکانس مشخصی وقفه ایجاد کند. کد مربوطه معمولاً به این شکل است :

```
...  
lapicw(TDCR, X1);  
lapicw(TIMER, PERIODIC | (T_IRQ0 + IRQ_TIMER));  
lapicw(TICR, 10000000);  
  
// Disable logical interrupt lines.  
lapicw(LINT0, MASKED);  
lapicw(LINT1, MASKED);
```

مقدار **TICR** تعیین‌کننده دوره تناوب وقفه است. در **xv6** اصلی، این مقدار معمولاً برای تولید وقفه هر ۱۰ میلی‌ثانیه تنظیم می‌شود. با اضافه کردن **cprintf** پس از **++ticks** در تابع **trap** (دسته‌بندی وقفه تایمر)، می‌توان مشاهده کرد که در هر ثانیه حدود ۱۰۰ بار چاپ انجام می‌شود که تأییدکننده این دوره تناوب است.

این فرکانس ثابت باعث می‌شود زمان‌بند بتواند هر ۱۰ میلی‌ثانیه کنترل را به دست گرفته و در صورت نیاز پردازش‌های **RUNNABLE** دیگر را اجرا کند.

11- با توجه به توضیحات داده شده، چه تابعی منجر به انجام شدن گذار **interrupt** در شکل ۱ خواهد شد؟

در **xv6**، گذار از حالت **RUNNING** به **RUNNABLE** در اثر وقفه (به ویژه وقفه تایمر) توسط تابع **yield()** انجام می‌شود. این تابع در فایل **proc.c** تعریف شده و در پاسخ به وقفه تایمر (و دیگر وقفه‌های مربوط به زمان‌بندی) فراخوانی می‌شود.

شرح عملکرد: وقفه تایمر باعث فراخوانی **trap()** می‌شود (**Trap**). نوع وقفه را تشخیص داده و اگر از نوع تایمر باشد، تابع **yield()** را فراخوانی می‌کند. **yield()** وضعیت پردازش جاری را از **RUNNING** به **RUNNABLE** تغییر داده، قفل‌های لازم را مدیریت کرده و **scheduler()** را فراخوانی می‌کند تا پردازش بعدی انتخاب شود.

بدین ترتیب، **yield()** عامل اصلی ایجاد گذار از حالت اجرا به حالت آماده در اثر وقفه است.

12- با توجه به توضیحات قسمت **scheduler dispatcher** می‌دانیم زمان‌بندی در **xv6** به شکل نوبت‌گردشی است. حال با توجه مشاهدات خود در این قسمت، استدلال کنید، کوانتوم زمانی این پیاده‌سازی از زمان‌بندی نوبت‌گردشی چند میلی‌ثانیه است؟

کوانتوم زمانی در **xv6** معادل ۱۰ میلی‌ثانیه است. وقفه تایمر در **xv6** هر ۱۰ میلی‌ثانیه یکبار صادر می‌شود (مطابق پاسخ سوال ۱۰). در هر وقفه تایمر، تابع **yield()** فراخوانی می‌شود که پردازش در حال اجرا را متوقف کرده و به حالت **RUNNABLE** برمی‌گرداند. سپس زمان‌بند (**scheduler**) پردازش بعدی را از میان پردازش‌های **RUNNABLE** انتخاب می‌کند.

از آنجا که وقفه تایمر تنها مکانیزم پیش‌گیری (**preemption**) در **xv6** است، هر پردازش حداکثر می‌تواند ۱۰ میلی‌ثانیه به صورت پیوسته اجرا شود، مگر اینکه زودتر به دلیلی مانند انتظار برای I/O، خود CPU را رها کند. این مدت زمان، همان کوانتوم زمانی در الگوریتم نوبت‌گردشی **xv6** است.

13-نتایج wait در نهایت از چه تابعی برای منتظر ماندن برای اتمام کار یک پردازش استفاده می‌کند؟

تابع wait در xv6 (که در فایل sysproc.c تعریف شده) برای انتظار به پایان رسیدن یک پردازش فرزند، در نهایت از تابع sleep () که در proc.c قرار دارد) استفاده می‌کند. پردازش والد با فراخوانی sleep، خودش را در حالت SLEEPING قرار می‌دهد و تا زمانی که یکی از فرزندان به حالت ZOMBIE برود، منتظر می‌ماند. پس از اعلام پایان فرزند، والد توسط تابع wakeup بیدار می‌شود.

14-با توجه به پاسخ سوال قبل، استفاده(های) دیگر این تابع چیست؟ (ذکر یک نمونه کافی است.)

تابع sleep در xv6 به طور کلی برای همگام‌سازی و انتظار برای رویدادها به کار می‌رود. یک نمونه دیگر از کاربرد آن، منتظر ماندن برای ورودی از صفحه‌کلید است. در راه‌اندازی کنسول، پردازش می‌تواند با فراخوانی sleep، تا زمانی که کاراکتری برای خواندن موجود نباشد، منتظر بماند.

15-الف) چه تابعی در سطح کرنل، منجر به آگاه‌سازی پردازش از رویدادی است که برای آن منتظر بوده است؟

ب) این تابع منجر به گذار از چه وضعیتی به چه وضعیتی در شکل ۱ خواهد شد؟
ج) آیا توابع دیگری وجود دارند که منجر به انجام این گذار شود؟ نام ببرید.

الف) تابع wakeup در فایل proc.c مسئول بیدار کردن پردازش‌هایی است که برای یک رویداد خاص در حال خواب (SLEEPING) هستند. این تابع با جستجو در جدول پردازش‌ها، تمام پردازش‌های در حال خواب روی کانال (chan) مشخص را بیدار می‌کند.

ب) تابع wakeup باعث گذار پردازش از حالت SLEEPING (منتظر) به حالت RUNNABLE (آماده اجرا) در دیاگرام حالت‌های پردازش (شکل ۱) می‌شود.

ج) بله، توابع دیگری نیز می‌توانند این گذار را ایجاد کنند. از جمله:

- exit هنگام پایان یک پردازش، والد را (اگر در حال انتظار باشد) بیدار می‌کند.
- kill اگر پردازش هدف در حال خواب باشد، آن را بیدار می‌کند (هرچند معمولاً باعث خاتمه پردازش می‌شود).
- pipeclose هنگام بستن یک پایپ، پردازش‌های در حال خواب روی آن پایپ را بیدار می‌کند.

16-در رابطه با پردازش‌های یتیم (Orphan) در xv6 آشنا شدید. رویکرد xv6 در بخش ۳.۳.۲ منبع درس نسبت به پردازش‌های یتیم چیست؟

در xv6، پردازش‌های یتیم (Orphan Processes) به پردازش‌هایی گفته می‌شود که والد آنها قبل از پایان یافتن خود پردازش، خاتمه یافته یا از بین رفته است. اگر پردازش‌های والد خود را از دست بدهد و هیچ پردازش‌ای برای جمع‌آوری وضعیت خروجی آن از طریق wait وجود نداشته باشد، ممکن است برای همیشه در حالت زامبی (ZOMBIE) باقی بماند و منابع آن هرگز آزاد نشود.

رویکرد xv6 برای مدیریت پردازش‌های یتیم:

xv6 این مشکل را با واگذاری خودکار پردازش‌های یتیم به پردازش شماره ۱ (init) حل می‌کند. هنگامیکه یک پردازش والد خاتمه می‌یابد در تابع exit()، فایل proc.c سیستم‌عامل بررسی می‌کند که آیا این پردازش، فرزند زنده دارد یا خیر. اگر

مجید سپاهی 810100163
امیرحسین صباحی 810101556

فرزندی داشته باشد، والد آن فرزندان به `init process` تغییر داده می‌شود. پردازش `init` به طور مداوم در حلقه‌ای قرار دارد که تابع `wait()` را فراخوانی می‌کند. بنابراین، هر پردازش یتیمی که در نهایت خاتمه یابد، توسط `init` شناسایی شده و وضعیت خروجی آن جمع‌آوری می‌شود. بدین ترتیب از انباشته شدن پردازش‌های زامبی در سیستم جلوگیری می‌شود.

4-زمان بندی ناهمگون:

تعریف نوع هسته در `proc.h`:

```
C proc.h > ...  
1  #define CORE_E 0  
2  #define CORE_P 1  
3
```

```
// Common CPU setup code.  
static void  
mpmain(void)  
{  
    int id = cpuid();  
    struct cpu *c = mycpu();  
    if(id % 2 == 0)  
        c->coretype = CORE_E;  
    else  
        c->coretype = CORE_P;  
  
    cprintf("cpu%d: starting as %s-core\n",  
           id,  
           (c->coretype == CORE_E) ? "E" : "P");  
  
    idtinit();          // load idt register  
    xchg(&(mycpu()->started), 1); // tell startothers() we're up  
    scheduler();        // start running processes  
}
```

باید جایی که هر CPU استارت می‌خورد، نوعش رو مشخص کنیم. بهترین جا تو `xv6` همین `mpmain()` تو `main.c` هست (ورود هر CPU به کرنل)

خروجی:


```
Booting from Hard Disk...
cpu1: starting as P-core
cpu2: starting as E-core
cpu3: starting as P-core
cpu0: starting as E-core
```

4.1. نوبت گردشی:

```
C param.h > NPROC
1  #define NPROC      64 // maximum number of processes
2  #define KSTACKSIZE 4096 // size of per-process kernel stack
3  #define NCPU       8 // maximum number of CPUs
4  #define NOFILE     16 // open files per process
5  #define NFILE      100 // open files per system
6  #define NINODE      50 // maximum number of active i-nodes
7  #define NDEV        10 // maximum major device number
8  #define ROOTDEV     1 // device number of file system root disk
9  #define MAXARG      32 // max exec arguments
10 #define MAXOPBLOCKS 10 // max # of blocks any FS op writes
11 #define LOGSIZE      (MAXOPBLOCKS*3) // max data blocks in on-disk log
12 #define NBUF         (MAXOPBLOCKS*3) // size of disk block cache
13 #define FSSIZE       1000 // size of file system in blocks
14 #define QUANTUM_TICKS 3
15
16
```

تعریف کوانتوم بر حسب تیک در param.h مقدار 3 از تقسیم 30 بر 3 حاصل می شود.

```
49  switch(tf->trapno){
50  case T_IRQ0 + IRQ_TIMER:
51      if(cpuid() == 0){
52          acquire(&tickslock);
53          ticks++;
54          wakeup(&ticks);
55          release(&tickslock);
56      }
57      struct proc *p = myproc();
58      if(p && p->state == RUNNING){
59          p->qticks++;
60          cprintf("tick %d: pid=%d, qticks=%d\n", ticks, p->pid, p->qticks);
61      }
62      lapiceoi();
63      break;
```

در case timer در trap.c همزمان با افزایش ticks ، تعداد تیک های کوانتوم پردازش در حال اجرا رو هم زیاد کرده و pid رو چاپ کنیم.

```
107 // (If it is still executing in the kernel, let it keep running
108 // until it gets to the regular system call return.)
109 if(myproc() && myproc()->killed && (tf->cs&3) == DPL_USER)
110     exit();
111
112 // Force process to give up CPU on clock tick.
113 // If interrupts were on while locks held, would need to check nlock.
114 if(myproc() && myproc()->state == RUNNING &&
115     tf->trapno == T_IRQ0+IRQ_TIMER){
116     if(myproc()->qticks >= QUANTUM_TICKS){
117         yield();
118     }
119 }
120
121
122 // Check if the process has been killed since we yielded
123 if(myproc() && myproc()->killed && (tf->cs&3) == DPL_USER)
124     exit();
125 }
```

در trap.c در جایی که هر تیک yield() می‌کند. باید شرط «رسیدن به کوانتوم» رو اضافه کنیم.

```
C busy.c > ...
1  #include "types.h"
2  #include "stat.h"
3  #include "user.h"
4
5  int
6  main(int argc, char *argv[])
7  {
8      int n = 3;
9      int i, j, pid;
10
11     if(argc > 1)
12         n = atoi(argv[1]);
13
14     for(i = 0; i < n; i++){
15         pid = fork();
16         if(pid < 0){
17             printf(1, "fork failed\n");
18             exit();
19         }
20         if(pid == 0){
21             for(j = 0; j < 100000000; j++){
22             }
23             exit();
24         }
25     }
26
27     for(i = 0; i < n; i++)
28         wait();
29
30     exit();
31 }
32
```

busy.c برنامه سطح کاربری که درون خود چند پردازش ایجاد می‌کند و هر پردازش مشغول انجام عملیاتی می‌شود که به اندازه کافی طول میکشد.

```
23 void
24 scheduler(void)
25 {
26     struct cpu *c = mycpu();
27     c->proc = 0;
28
29     for(;;){
30         sti();
31
32         acquire(&ptable.lock);
33
34         struct proc *p = 0;
35         struct proc *toRun = 0;
36         int i;
37
38         for(i = 0; i < NPROC; i++){
39             int idx = (last_idx + i) % NPROC;
40             p = &ptable.proc[idx];
41
42             if(p->state != RUNNABLE)
43                 continue;
44
45             toRun = p;
46             last_idx = (idx + 1) % NPROC;
47             break;
48         }
49
50         if(toRun){
51             c->proc = toRun;
52             switchvm(toRun);
53             toRun->state = RUNNING;
54
55             toRun->qticks = 0;
56
57             switch(&c->scheduler, toRun->context):
58                 switchkvm();
59
60             c->proc = 0;
61         }
62
63         release(&ptable.lock);
64     }
65 }
```

در proc.c تابع scheduler به شکل نوبت‌گردشی بازنویسی می‌کنیم با استفاده از last_idx (نگهداری آخرین اندیس پردازش اجراشده) و صفرکردن qticks در شروع هر کوانتوم

خروجی:

```
Booting from Hard Disk...
cpu0: starting as E-core
sb: size 1000 nblocks 941 ninodes 200 nlog 30 logstart 2 inodestart 32 bmap sta
t 58
tick 2: pid=1, qticks=1
tick 3: pid=1, qticks=1
intick 4: pid=1, qticks=1
it: starting sh
$ busy 3
tick 5004: pid=3, qticks=1
tick 5005: pid=3, qticks=1
$ _
```

4.2 جدا کردن صف هر هسته (امتیازی):

```

409 scheduler(void)
415 for(;;){
424 if(c->coretype == CORE_P){
426     for(i = 0; i < NPROC; i++){
427         p = &ptable.proc[i];
428         if(p->state != RUNNABLE)
429             continue;
430         if(p->homecpu != id)
431             continue;
432         if(p->ctime < min_ctime){
433             min_ctime = p->ctime;
434             toRun = p;
435         }
436     }
437 } else {
438     for(i = 0; i < NPROC; i++){
439         int idx = (last_idx + i) % NPROC;
440         p = &ptable.proc[idx];
441
442         if(p->state != RUNNABLE)
443             continue;
444         if(p->homecpu != id)
445             continue;
446
447         toRun = p;
448         last_idx = (idx + 1) % NPROC;
449         break;
450     }
451 }
452
453 if(toRun){
454     cprintf("SCHED: cpu%d(core=%s) running pid=%d (%s), homecpu=%d\n",
455            id,
456            c->coretype == CORE_E ? "E" : "P",
457            toRun->pid,
458            toRun->name,
459            toRun->homecpu);
460

```

در proc.c پس از مقداردهی homecpu در allocproc / userinit / fork تابع scheduler با اضافه کردن فیلد homecpu در ساختار proc، صف منطقی جداگانه‌ای برای هر هسته تعریف شد؛ به این صورت که هر پردازنده تنها پردازنده‌هایی را که homecpu آن‌ها با cpuid خود برابر است، زمان‌بندی می‌کند. همچنین تستی می‌نویسیم برای چاپ homecpu داخل scheduler.

در خروجی یا فقط روی cpu0 پردازنده‌هایی می‌بینیم که homecpu=0 دارند. یا فقط روی cpu1 پردازنده‌هایی می‌بینیم که homecpu=1 دارند.

```

cpu0 (E-core) tick 8: pid=1, qticks=1
SCHED: cpu0(core=E) running pid=1 (init), homecpu=0
SCHED: cpu0(core=E) running pid=1 (init), homecpu=0
SCHED: cpu0(core=E) running pid=1 (init), homecpu=0
SCHED: cpu0(core=E) running pid=1 (init), homecpu=0
SCHED: cpu0(core=E) running pid=1 (init), homecpu=0
SCHED: cpu0(core=E) running pid=1 (init), homecpu=0
SCHED: cpu0(core=E) running pid=1 (init), homecpu=0

```

```
// For process struct
struct proc {
    uint sz;                // Size of process memory (bytes)
    pde_t* pgdir;           // Page table
    char *kstack;           // Bottom of kernel stack for this process
    enum procstate state;   // Process state
    int pid;                // Process ID
    struct proc *parent;    // Parent process
    struct trapframe *tf;   // Trap frame for current syscall
    struct context *context; // swtch() here to run process
    void *chan;             // If non-zero, sleeping on chan
    int killed;             // If non-zero, have been killed
    struct file *ofile[NOFILE]; // Open files
    struct inode *cwd;      // Current directory
    char name[16];          // Process name (debugging)
    int qticks;
    uint ctime;
    int homecpu;
};
```

در proc.h زمان ایجاد پردازش (ctime) رو در struct proc ذخیره می کنیم.

در proc.c دوباره در تابع scheduler برای اجرای fcfs به پردازشها بر اساس زمان ایجاد (ctime) ترتیب می دهیم. این کار فقط برای P-core ها انجام می شود، چرا که برای E-core ها همچنان از Round Robin استفاده می کنیم.

```
if(c->coretype == CORE_P){
    uint min_ctime = ~0;
    for(i = 0; i < NPROC; i++){
        p = &ptable.proc[i];
        if(p->state == RUNNABLE && p->ctime < min_ctime){
            min_ctime = p->ctime;
            toRun = p;
        }
    }
} else {
    for(i = 0; i < NPROC; i++){
        int idx = (last_idx + 1) % NPROC;
        p = &ptable.proc[idx];

        if(p->state != RUNNABLE)
            continue;

        toRun = p;
        last_idx = (idx + 1) % NPROC;
        break;
    }
}
```

خروجی:


```
cpu1: starting as P-core
cpu0: starting as E-core
sb: size 1000 nblocks 941 ninodes 200 nlog 30 logstart 2 inodestart 32 bmap start 58
cpu0 (E-core) tick 2: pid=1, qticks=1
cpu0 (E-core) tick 3: pid=1, qticks=1
cpu0 (E-core) tick 4: pid=1, qticks=1
init: starting sh
$ busy 8
cpu0 (E-core) tick 1613: pid=3, qticks=1
cpu0 (E-core) tick 1614: pid=3, qticks=2
$ busy 8
cpu0 (E-core) tick 2249: pid=12, qticks=1
cpu1 (P-core) tick 2249: pid=15, qticks=1
cpu0 (E-core) tick 2250: pid=12, qticks=2
cpu1 (P-core) tick 2250: pid=20, qticks=1
$ busy 8
cpu0 (E-core) tick 5027: pid=21, qticks=1
cpu0 (E-core) tick 5028: pid=21, qticks=2
cpu1 (P-core) tick 5028: pid=27, qticks=1
cpu0 (E-core) tick 5029: pid=21, qticks=3
cpu1 (P-core) tick 5029: pid=21, qticks=1
$
```

4.4 جابه جایی پردازش میان صف ها:

در proc.c تابع شمارش طول صف هر CPU رو اضافه می کنیم:

```
26 static int queue_len(int cpu_id)
27 {
28     int n = 0;
29     struct proc *p;
30     for(p = ptable.proc; p < &ptable.proc[NPROC]; p++){
31         if(p->state == RUNNABLE && p->homecpu == cpu_id)
32             n++;
33     }
34     return n;
35 }
```

در proc.c در این مرحله، E-core باید اگر بارش زیاد بود (یعنی تعداد پردازشها بیشتر از P-core ها بود)، اولین پردازش اضافه شده به صف خود را به P-core هل دهد. برای این منظور تابعی به نام push_from_ecore می نویسیم.

مجید سپاهی 810100163
امیرحسین صباخی 810101556

```
37 static void push_from_ecore(int srcid)
38 {
39     int i;
40     struct proc *p, *victim = 0;
41     uint victim_ctime = 0xffffffff;
42
43     int min_p_len = 0x3fffffff;
44     int target_p = -1;
45
46     for(i = 0; i < ncpu; i++){
47         if(cpus[i].coretype != CORE_P)
48             continue;
49
50         int qlen = queue_len(i);
51         if(qlen < min_p_len){
52             min_p_len = qlen;
53             target_p = i;
54         }
55     }
56
57     if(target_p < 0)
58         return;
59
60     int src_len = queue_len(srcid);
61
62     if(src_len < min_p_len + 3)
63         return;
64
65     for(p = ptable.proc; p < &ptable.proc[NPROC]; p++){
66         if(p->state != RUNNABLE)
67             continue;
68         if(p->homecpu != srcid)
69             continue;
70         if(strncmp(p->name, "init", 4) == 0)
71             continue;
72         if(strncmp(p->name, "sh", 2) == 0)
73             continue;
74
75         if(victim == 0 || p->ctime < victim_ctime){
76             victim = p;
77             victim_ctime = p->ctime;
78         }
79     }
80     if(victim){
81         victim->homecpu = target_p;
82         cprintf("PUSH: pid-%d from cpu%d to cpu%d\n",
83             victim->pid, srcid, target_p);
84     }
```

در proc.c می‌خواهیم هر ۵۰ میلی‌ثانیه (معادل ۵ تیک) این بررسی را انجام دهیم. برای این کار باید یک تابع به نام maybe_push ایجاد کنیم که هر ۵۰ms یکبار پردازنده‌ها را بررسی کند و آن را در trap.c صدا می‌زنیم:

```
88 void maybe_push(void)
89 {
90     int id = cpuid();
91     struct cpu *c = mycpu();
92
93     if(c->coretype != CORE_E)
94         return;
95
96     if(ticks - lastpush[id] < 5)
97         return;
98
99     lastpush[id] = ticks;
100
101     acquire(&ptable.lock);
102     push_from_ecore(id);
103     release(&ptable.lock);
104 }
105
```

```
$ busy 25
SCHED: cpu0(core=E) running pid=2 (sh), homecpu=0
SCHED: cpu0(core=E) running pid=15 (sh), homecpu=0
cpu0 (E-core) tick 3436: pid=15, qticks=1
cpu0 (E-core) tick 3437: pid=15, qticks=2
cpu0 (E-core) tick 3438: pid=15, qticks=3
SCHED: cpu0(core=E) running pid=16 (busy), homecpu=6
SCHED: cpu0(core=E) running pid=17 (busy), homecpu=6
SCHED: cpu0(core=E) running pid=18 (busy), homecpu=6
SCHED: cpu0(core=E) running pid=15 (busy), homecpu=6
cpu0 (E-core) tick 3439: pid=15, qticks=1
cpu0 (E-core) tick 3440: pid=15, qticks=2
PUSH: pid=19 from cpu0 to cpu1
SCHED: cpu1(core=P) running pid=19 (busy), homecpu=1
```

در خروجی می بینیم برخی از مواقع پردازنده‌ها به P-core منتقل می‌شوند.

5.4 ارزیابی الگوریتم های زمانبندی:

```
C proc.c > ...
1  #include "types.h"
2  #include "defs.h"
3  #include "param.h"
4  #include "memlayout.h"
5  #include "mmu.h"
6  #include "x86.h"
7  #include "proc.h"
8  #include "defs.h"
9  #include "spinlock.h"
10
11  struct {
12      struct spinlock lock;
13      struct proc proc[NPROC];
14  } ptable;
15
16  static int last_idx = 0;
17  extern uint ticks;
18  int th_active = 0;
19  int th_done = 0;
20  uint th_start = 0;
21  static uint lastpush[NCPU];
22  static struct proc *initproc;
```


مجید سپاهی 810100163
امیرحسین صباخی 810101556

در proc.c متغیرهای سراسری برای اندازه‌گیری را تعریف می‌کنیم همچنین در تابع exit() هر وقت یک پردازش تمام می‌شود و exit می‌زند، اگر اندازه‌گیری روشن باشد، یک واحد به شمارنده اضافه می‌کنیم.

در proc.c دو تابع کرنلی برای شروع/پایان اندازه‌گیری می‌نویسیم در defs.h در بخش proc.c این دو تابع را اعلان می‌کنیم:

```
406 void
407 start_throughput(void)
408 {
409     acquire(&tickslock);
410     th_active = 1;
411     th_done = 0;
412     th_start = ticks;
413     release(&tickslock);
414 }
415
416 void
417 stop_throughput(void)
418 {
419     uint start, end;
420     int done, thr;
421
422     acquire(&tickslock);
423     if(!th_active){
424         release(&tickslock);
425         return;
426     }
427     th_active = 0;
428     start = th_start;
429     done = th_done;
430     end = ticks;
431     release(&tickslock);
432
433     if(end <= start){
434         cprintf("throughput: duration too short (ticks=%d)\n", end - start);
435         return;
436     }
437
438     thr = (done * 100) / (end - start);
439
440     cprintf("throughput: %d procs/sec (finished=%d, ticks=%d)\n",
441           thr, done, end - start);
442 }
443
```

```
80 int
81 sys_th_start(void)
82 {
83     start_throughput();
84     return 0;
85 }
86
87 int
88 sys_th_stop(void)
89 {
90     stop_throughput();
91     return 0;
92 }
93
```

مجید سپاهی 810100163
امیرحسین صباحی 810101556

پیداسازی دو syscall در sysproc.c در syscall.h هم شماره syscall ها را اضافه می کنیم در syscall.c هم دو ورودی جدید به جدول syscall ها اضافه می کنیم. در user.h و usys.s wrapper های user-space را اضافه می کنیم:

```
C user.h > strcpy(char *, const char *)
1 struct stat;
2 struct rtdat;
3
4 // system calls
5 int fork(void);
6 int exit(void) __attribute__((noreturn));
7 int wait(void);
8 int pipe(int*);
9 int write(int, const void*, int);
10 int read(int, void*, int);
11 int close(int);
12 int kill(int);
13 int exec(char*, char**);
14 int open(const char*, int);
15 int mknod(const char*, short, short);
16 int unlink(const char*);
17 int fstat(int fd, struct stat*);
18 int link(const char*, const char*);
19 int mkdir(const char*);
20 int chdir(const char*);
21 int dup(int);
22 int getpid(void);
23 char* sbrk(int);
24 int sleep(int);
25 int uptime(void);
26 int th_start(void);
27 int th_stop(void);
```

thtest.c برنامه تست کاربری فراخوانی سیستمی:

```
C thtest.c > main(int, char* [])
1 #include "types.h"
2 #include "stat.h"
3 #include "user.h"
4
5 int
6 main(int argc, char *argv[])
7 {
8     int n = 20;
9     int i, pid;
10
11     if(argc > 1)
12         n = atoi(argv[1]);
13
14     th_start();
15
16     for(i = 0; i < n; i++){
17         pid = fork();
18         if(pid < 0){
19             printf(1, "fork failed\n");
20             exit();
21         }
22         if(pid == 0){
23             int j;
24             for(j = 0; j < 100000000; j++)
25                 ;
26             exit();
27         }
28     }
29
30     while(wait() > 0)
31         ;
32
33     th_stop();
34
35     exit();
36 }
```

در خروجی گذردهی به صورت «تعداد پردازنده‌های خاتمه‌یافته در واحد زمان» در نظر گرفته شده است. برای اندازه‌گیری، در بازه‌ی بین دو فراخوانی سیستمی، تعداد پردازنده‌هایی که فراخوانی `exit` را انجام می‌دهند شمرده می‌شود و با تقسیم این تعداد بر مدت‌زمان طی‌شده (بر حسب ثانیه)، مقدار گذردهی به‌دست می‌آید. با توجه به این‌که در `xv6` هر ۱۰۰ تیک تقریباً معادل یک ثانیه است، گذردهی به صورت $(finished * 100)$

`delta_ticks` محاسبه شده است (31=ticks, 50=finished) `throughput: 161 procs/sec`
 $50 * 100 / 31$

```
PUSH: pid=50 from cpu0 to cpu1
SCHED: cpu1(core=P) running pid=50 (thtest), homecpu=1
cpu0 (E-core) tick 2396: pid=3, qticks=3
SCHED: cpu0(core=E) running pid=51 (thtest), homecpu=0
SCHED: cpu0(core=E) running pid=52 (thtest), homecpu=0
SCHED: cpu0(core=E) running pid=53 (thtest), homecpu=0
SCHED: cpu0(core=E) running pid=3 (thtest), homecpu=0
cpu0 (E-core) tick 2397: pid=3, qticks=1
cpu0 (E-core) tick 2398: pid=3, qticks=2
cpu0 (E-core) tick 2399: pid=3, qticks=3
SCHED: cpu0(core=E) running pid=3 (thtest), homecpu=0
cpu0 (E-core) tick 2400: pid=3, qticks=1
cpu0 (E-core) tick 2401: pid=3, qticks=2
throughput: 161 procs/sec (finished=50, ticks=31)
```

5.5 فراخوانیهای سیستمی مورد نیاز:

```
703
704 static char*
705 state_str(enum procstate s)
706 {
707     static char *states[] = {
708         [UNUSED]    "unused",
709         [EMBRYO]     "embryo",
710         [SLEEPING]   "sleep",
711         [RUNNABLE]   "runnable",
712         [RUNNING]    "running",
713         [ZOMBIE]     "zombie"
714     };
715
716     if(s < 0 || s > ZOMBIE)
717         return "???";
718     return states[s];
719 }
720
721 void
722 procdump_sched(void)
723 {
724     struct proc *p;
725
726     acquire(&table.lock);
727     cprintf("name\tpid\tstate\tcpu\talgo\tlifetime_ticks\n");
728     for(p = table.proc; p < &table.proc[NPROC]; p++){
729         if(p->state == UNUSED)
730             continue;
731
732         char *st = state_str(p->state);
733         int cpu = p->homecpu;
734         char *algo = (cpus[cpu].coretype == CORE_E) ? "RR" : "FCFS";
735         uint lifetime = ticks - p->ctime;
736
737         cprintf("%s\t%d\t%s\t%d\t%s\t%d\n",
738             p->name, p->pid, st, cpu, algo, lifetime);
739     }
740     release(&table.lock);
741 }
```

در انتهای proc.c یک تابع کرنلی برای چاپ اطلاعات پردازنده‌ها می‌نویسیم

```

94  int
95  sys_printinfo(void)
96  {
97      procdump_sched();
98      return 0;
99  }
100

```

همچنین سیستم با ایجاد و usys.s و 07:42

در sysproc.c بدنه «syscall چاپ اطلاعات» را می‌نویسیم همچنین مانند قسمت 5.4 همان مراحل را برای این فراخوانی سیستمی انجام می‌دهیم با ایجاد تغییرات در defs.h و syscall.c و syscall.h و user.h و usys.s.

خروجی:

```

name    pid    state  cpu    algo    lifetime_ticks
init     1     sleep   0     RR      15790
sh       2     sleep   0     RR      15777
st      88     running 1     FCFS    11307
cpu1 (P-core) tick 15790: pid=88, qticks=1
SCHED: cpu0(core=E) running pid=2 (sh), homecpu=0

```

6.5 برنامه‌های سطح کاربر:

```

C: gcc -f
11  busy_work(int k)
12  {
13      for(i = 0; i < k; i++){
14          //
15      }
16  }
17
18  int
19  main(int argc, char *argv[])
20  {
21      int n = 20;
22      int i, pid;
23
24      if(argc > 1)
25          n = atoi(argv[1]);
26
27      printf("schedtest: n=%d\n", n);
28
29      th_start();
30
31      for(i = 0; i < n; i++){
32          pid = fork();
33          if(pid < 0){
34              printf("fork failed\n");
35              exit(1);
36          }
37          if(pid == 0){
38              busy_work(5 + (i % 5));
39              exit(0);
40          }
41      }
42
43      printf("n--- after fork ---\n");
44      printfinfo();
45
46      busy_work(10);
47
48      printf("n--- mid-run ---\n");
49      printfinfo();
50
51      while(wait() > 0)
52          ;
53
54      th_stop();
55
56      printf("n--- after wait ---\n");
57      printfinfo();
58
59      exit(0);
60  }

```

برنامه تست st.c در برنامه‌های تست، به جای استفاده از sleep، از حلقه‌های تو در تو و محاسبات عددی استفاده شده است تا پردازنده‌ها به صورت مداوم در حالت قابل اجرا قرار گیرند و فشار واقعی بر زمان‌بند وارد شود. به عنوان نمونه،

مجید سپاهی 810100163

امیرحسین صباحی 810101556

در تابع busy_work دو حلقه‌ی تو در تو همراه با عملیات جمع ساده روی یک متغیر volatile اجرا می‌شود. هر پردازش‌ی فرزند تعداد متفاوتی تکرار (پارامتر k) را انجام می‌دهد تا طول اجرای پردازش‌ها متفاوت بوده و رفتار الگوریتم‌های RR و FCFS و همچنین مکانیزم توازن بار به‌صورت ملموس قابل مشاهده و ارزیابی باشد.

خروجی اطلاعات مورد نیاز در هر مقطع:

Cpus=1

name	pid	state	cpu	algo	lifetime_ticks
init	1	sleep	0	RR	11685
sh	2	sleep	0	RR	11677
st	67	running	0	RR	3240
st	68	zombie	0	RR	3236
st	69	zombie	0	RR	3236
st	70	zombie	0	RR	3236
st	71	zombie	0	RR	3236
st	72	zombie	0	RR	3226
st	73	zombie	0	RR	3226
st	74	zombie	0	RR	3226
st	75	zombie	0	RR	3226
st	76	zombie	0	RR	3226
st	77	zombie	0	RR	3226

```

=== after fork ===
cpu0 (E-core) tick 7825: pid=70, qticks=2
cpu1 (P-core) tick 7825: pid=66, qticks=2
name  pid  state  cpu  algo  lifetime_ticks
init   1    sleep  0    RR    7825
sh     2    sleep  0    RR    7814
schedtest 66    running 1    FCFS    30
schedtest 67    runnable 0    RR    26
schedtest 68    runnable 0    RR    26
schedtest 69    runnable 0    RR    25
schedtest 70    running 0    RR    25
schedtest 71    runnable 1    FCFS    25
schedtest 72    runnable 1    FCFS    21
schedtest 73    runnable 1    FCFS    20
schedtest 74    runnable 1    FCFS    20
schedtest 75    runnable 1    FCFS    20
schedtest 76    runnable 1    FCFS    19
schedtest 77    runnable 1    FCFS    19
schedtest 78    runnable 1    FCFS    19
schedtest 79    runnable 1    FCFS    19
schedtest 80    runnable 1    FCFS    18
schedtest 81    runnable 1    FCFS    18
schedtest 82    runnable 1    FCFS    17
schedtest 83    runnable 1    FCFS    17
schedtest 84    runnable 1    FCFS    17
schedtest 85    runnable 1    FCFS    17
schedtest 86    runnable 1    FCFS    16
schedtest 87    runnable 1    FCFS    16
schedtest 88    runnable 1    FCFS    16
schedtest 89    runnable 1    FCFS    14
schedtest 90    runnable 1    FCFS    14
schedtest 91    runnable 1    FCFS    14
schedtest 92    runnable 1    FCFS    13
schedtest 93    runnable 1    FCFS    13
schedtest 94    runnable 1    FCFS    13
schedtest 95    runnable 1    FCFS    13
schedtest 96    runnable 1    FCFS    13
schedtest 97    runnable 1    FCFS    11
schedtest 98    runnable 1    FCFS    11
schedtest 99    runnable 1    FCFS    10
schedtest 100   runnable 1    FCFS    10
schedtest 101   runnable 1    FCFS    10
schedtest 102   runnable 1    FCFS    10
schedtest 103   runnable 1    FCFS    8
schedtest 104   runnable 1    FCFS    8
schedtest 105   runnable 1    FCFS    8
schedtest 106   runnable 1    FCFS    7
schedtest 107   runnable 1    FCFS    7
schedtest 108   runnable 1    FCFS    7
schedtest 109   runnable 1    FCFS    5
schedtest 110   runnable 1    FCFS    5

```

خروجی اطلاعات مورد نیاز در هر مقطع: cpus=2

```

throughput: 4 procs/sec (finished=50, ticks=11982)
cpu1 (P-core) tick 14849: pid=24, qticks=1
=== after cpu1 (P-core) tick 14850: pid=24, qticks=2
wait ===
name  pid  state  cpu  algo  lifetime_ticks
init   1    sleep  0    RR    14851
sh     2    sleep  0    RR    14837
st     24    running 1    FCFS    11991
cpu1 (P-core) tick 14851: pid=24, qticks=3
SCHED: cpu1(core=P) running pid=24 (st), homecpu=1
SCHED: cpu0(core=E) running pid=2 (sh), homecpu=0
$

```

بررسی بارم کمترین و بیشترین بار در هر مقطع

خروجی که در نهایت گذردهی سیستم عامل را گزارش داده: 4 procs/sec